

MATLAB Error Identifiers Cheatsheet

Overview

Important Note: MathWorks does not provide an official comprehensive list of all built-in error identifiers. Unlike Python's well-documented exception hierarchy, MATLAB error identifiers are scattered throughout individual function documentation.

Error identifiers follow the format: `Component:Mnemonic` (e.g., `MATLAB:UndefinedFunction`)

Core Exception Handling

Main Class

- **MException** - Base class for all MATLAB exceptions
 - [Documentation](#)

Common Built-in Error Identifiers

General Function Errors

Error Identifier	Description	Trigger Example
<code>MATLAB:UndefinedFunction</code>	Function not found or not on path	<code>nonExistentFunc()</code>
<code>MATLAB:TooManyInputs</code>	Too many input arguments	Function called with extra args
<code>MATLAB:TooManyOutputs</code>	Too many output arguments	Requesting too many outputs
<code>MATLAB:minrhs</code>	Not enough input arguments	Function called with missing args
<code>MATLAB:maxrhs</code>	Too many input arguments	Same as TooManyInputs
<code>MATLAB:unassignedOutputs</code>	Function didn't assign outputs	Output variable not set
<code>MATLAB:narginchk:notEnoughInputs</code>	Insufficient inputs	<code>narginchk</code> validation fails
<code>MATLAB:nargoutchk:tooManyOutputs</code>	Too many outputs requested	<code>nargoutchk</code> validation fails

Array & Indexing Errors

Error Identifier	Description	Trigger Example
<code>MATLAB:badsubscript</code>	Invalid array indexing	<code>A(100)</code> when <code>A</code> has 10 elements
<code>MATLAB:dimsAgree</code>	Array dimension mismatch	<code>A + B</code> with incompatible sizes
<code>MATLAB:sizeDimensionsMustMatch</code>	Size dimensions must match	Matrix operation size mismatch
<code>MATLAB:subsassignndimmismatch</code>	Subscript assignment mismatch	<code>A(1,1) = [1 2 3]</code>
<code>MATLAB:badsubscript:IndexOutOfRange</code>	Index exceeds bounds	Array indexing beyond size

Type & Value Errors

Error Identifier	Description	Trigger Example
<code>MATLAB:invalidType</code>	Invalid data type	Wrong type passed to function
<code>MATLAB:expectedInteger</code>	Expected integer value	Non-integer where integer needed
<code>MATLAB:expectedNonNegative</code>	Expected non-negative value	Negative value not allowed
<code>MATLAB:notLessEqual</code>	Value constraint violation	Value exceeds maximum
<code>MATLAB:assertion:failed</code>	Assertion failed	<code>assert(false)</code>

Memory & System Errors

Error Identifier	Description	Trigger Example
<code>MATLAB:nomem</code>	Out of memory	Allocation fails due to memory
<code>MATLAB:interrupt</code>	User interrupted execution	User presses Ctrl+C

File I/O Errors

Error Identifier	Description	Trigger Example
<code>MATLAB:FileIO:InvalidFid</code>	Invalid file identifier	Bad file handle used
<code>MATLAB:fileIO:fileNotFound</code>	File not found	<code>fopen('nonexistent.txt')</code>

Other Common Errors

Error Identifier	Description	Trigger Example
<code>MATLAB:ambiguousSyntax</code>	Ambiguous syntax	Syntax parser confusion
<code>MATLAB:invalidInput</code>	Generic invalid input	Input validation fails

How to Discover Error Identifiers

Since there's no official complete list, use these methods:

Method 1: Try-Catch with MException

```
matlab

try
    % Code that causes error
    A = [1 2 3];
    B = A(10); % Index out of bounds
catch ME
    fprintf('Error ID: %s\n', ME.identifier);
    fprintf('Message: %s\n', ME.message);
end
```

Method 2: Using MException.last

```
matlab

% After an uncaught error occurs:
>> nonExistentFunction()
% Error appears...

>> ME = MException.last;
>> ME.identifier
ans = 'MATLAB:UndefinedFunction'
```

Method 3: Check Function Documentation

Some functions document their specific error identifiers in their reference pages.

Creating Custom Error Identifiers

```
matlab

% Format: 'Component:Mnemonic'
error('MyToolbox:InvalidInput', 'Input must be positive');

% With formatting
error('MyApp:RangeError', 'Value %d exceeds maximum of %d', val, maxVal);
```

Naming Rules

- Format: `Component:Mnemonic` (or `Component:SubComponent:Mnemonic`)
- Must start with a letter
- Can contain: A-Z, a-z, 0-9, underscores
- No whitespace allowed
- Example: `'MyToolbox:FileNotFound'`

Catching Specific Errors

```
matlab

try
    % Some operation
    someFunction();
catch ME
    switch ME.identifier
        case 'MATLAB:UndefinedFunction'
            warning('Function not found. Using default behavior.');
```

Official Documentation Links

Main References

- [MException Class](#) - Main exception class documentation

- [error Function](#) - Throwing errors with identifiers
- [Issue Warnings and Errors](#) - General error handling guide
- [Exception Handling](#) - Try-catch documentation

Related Functions

- [try, catch](#) - Exception handling structure
- [throw](#) - Throw exception
- [rethrow](#) - Rethrow exception
- [assert](#) - Throw error if condition false
- [warning](#) - Display warning message
- [lasterror](#) - Last error information (legacy)
- [lasterr](#) - Last error message (legacy)

Key Differences from Python

Feature	Python	MATLAB
Error Type	Exception Classes	String Identifiers
Hierarchy	Class inheritance	Colon-separated namespaces
Built-in List	Well documented	No official comprehensive list
Example	<code>ValueError</code> , <code>TypeError</code>	<code>'MATLAB:invalidType'</code> , <code>'MATLAB:badsubscript'</code>
Catching	<code>except ValueError:</code>	<code>if strcmp(ME.identifier, 'MATLAB:...')</code>

Tips

1. **Always use identifiers** in your errors for better error handling
2. **Be specific** with component names (e.g., `MyApp:FileIO:NotFound`)
3. **Document your identifiers** if building a toolbox
4. Use `MException.last` in interactive debugging
5. Check `ME.identifier` before deciding how to handle errors
6. **Consistent naming** helps with internationalization

Example: Complete Error Handling

matlab

```

function result = divideArrays(A, B)
    % Validate inputs
    if ~isnumeric(A) || ~isnumeric(B)
        error('MyMath:invalidType', 'Inputs must be numeric arrays');
    end

    if ~isequal(size(A), size(B))
        error('MyMath:sizeMismatch', ...
            'Arrays must have same size. A is %dx%d, B is %dx%d', ...
            size(A,1), size(A,2), size(B,1), size(B,2));
    end

    if any(B(:) == 0)
        error('MyMath:divisionByZero', 'Division by zero detected');
    end

    try
        result = A ./ B;
    catch ME
        % Log error details
        fprintf('Error in divideArrays:\n');
        fprintf('  ID: %s\n', ME.identifier);
        fprintf('  Message: %s\n', ME.message);
        rethrow(ME);
    end
end

```

Note: This cheatsheet is based on commonly encountered errors. For the most up-to-date and specific error identifiers, always check the individual function documentation or use the discovery methods described above.

Last Updated: Based on MATLAB documentation as of 2024-2025